Final Project Report

Project 2: Loud versus Quiet

Student Name	**Steve**
Project	Project 2 — Loud versus Quiet
Specialization	Network Scanning and Enumeration
Scanner Machine	Kali Linux (hostname: V2310) — 192.168.10.74
Target Machine	Windows Server VM (V2434 / STEVE) — 192.168.10.71
Tools Used	Nmap, Wireshark

1. Objective

Learn which scan types make noise and which stay quiet — and prove it with packets. Three scans (TCP Connect, SYN and UDP) are run against the same lab target, each scan is captured in Wireshark, and the packet trails and log footprints are compared to explain which scan an IDS (Intrusion Detection System) would catch first, and why.

Status: All three scans (TCP Connect, SYN and UDP) were performed against the same target and captured in Wireshark. Every screenshot is placed at its step in the Procedure, with an explanation under each one, and every packet count comes from the student's own captures.

2. Tools Used

Tool	Purpose
Kali Linux (VMware)	Scanner machine used to run all scans and captures
Nmap	Runs the three scan types: -sT (Connect), -sS (SYN), -sU (UDP)
Wireshark	Captures the packets of each scan on the eth0 interface so the trails can be counted and compared

3. Procedure

Setup check: before scanning, the target was pinged from Kali — 4 packets transmitted, 4 received, 0% packet loss (visible at the top of Figure 1). For every scan the same cycle was used: start the Wireshark capture on eth0 first, run the Nmap scan, stop the capture after Nmap finished, save the capture, then apply the display filter ip.addr == 192.168.10.71 and record the Displayed packet count.

Step 1 — TCP Connect scan (the loud scan). Capture saved as: capture_connect.pcapng

```
nmap -sT 192.168.10.71 -oN scan-connect.txt
```

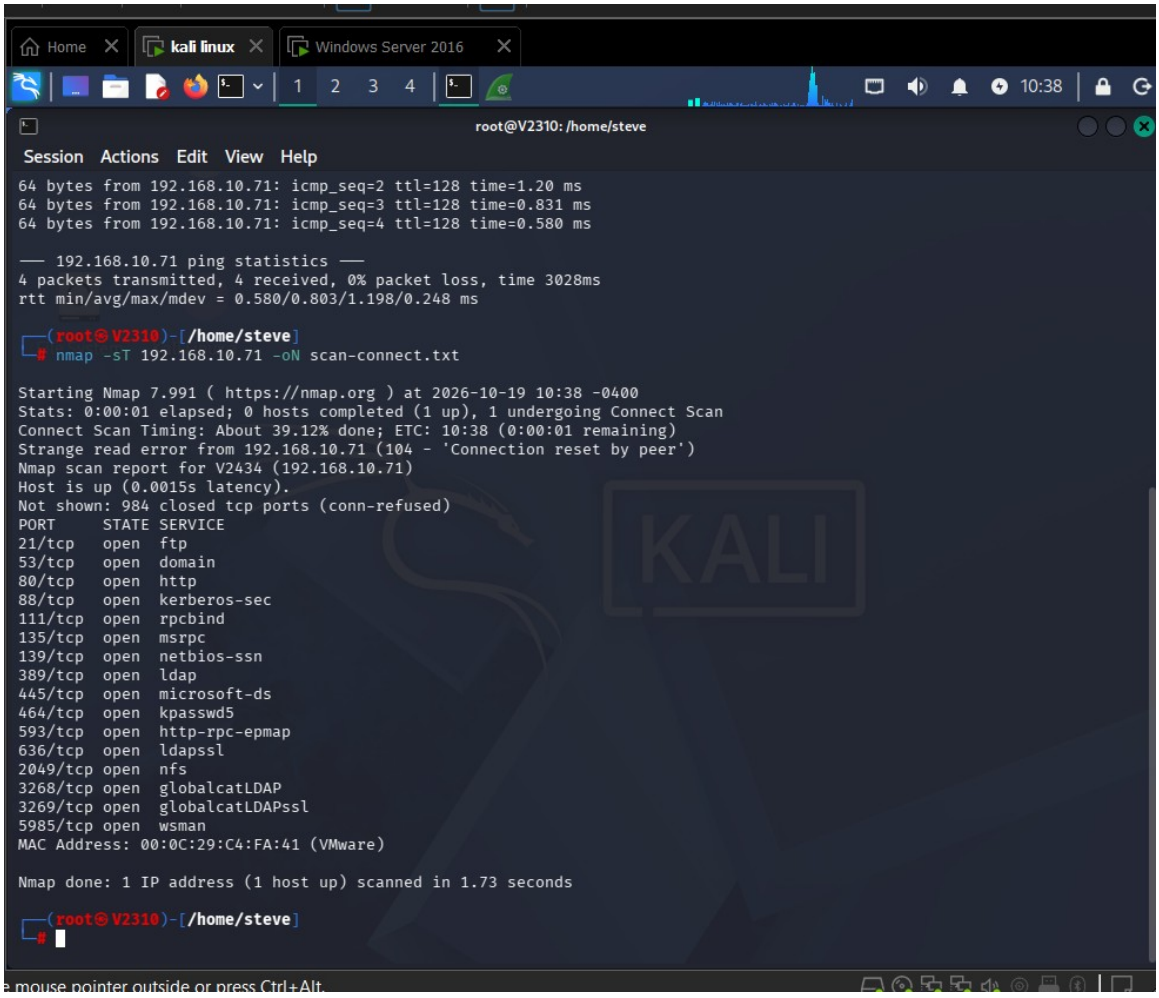


Figure 1 — Step 1 (terminal): ping check, then the TCP Connect scan result.

What this screenshot shows: The terminal result of the Connect scan. The ping replies at the top prove the target was reachable before scanning. The scan itself finished in only 1.73 seconds and found the same 16 open TCP ports as Project 1 (21, 53, 80, 88, 111, 135, 139, 389, 445, 464, 593, 636, 2049, 3268, 3269, 5985), with 984 ports closed. One line is important evidence for this project: Nmap reported a 'Connection reset by peer' read error from the target — the target's services were actively reacting to the completed connections, which is exactly the kind of footprint a loud scan leaves behind.

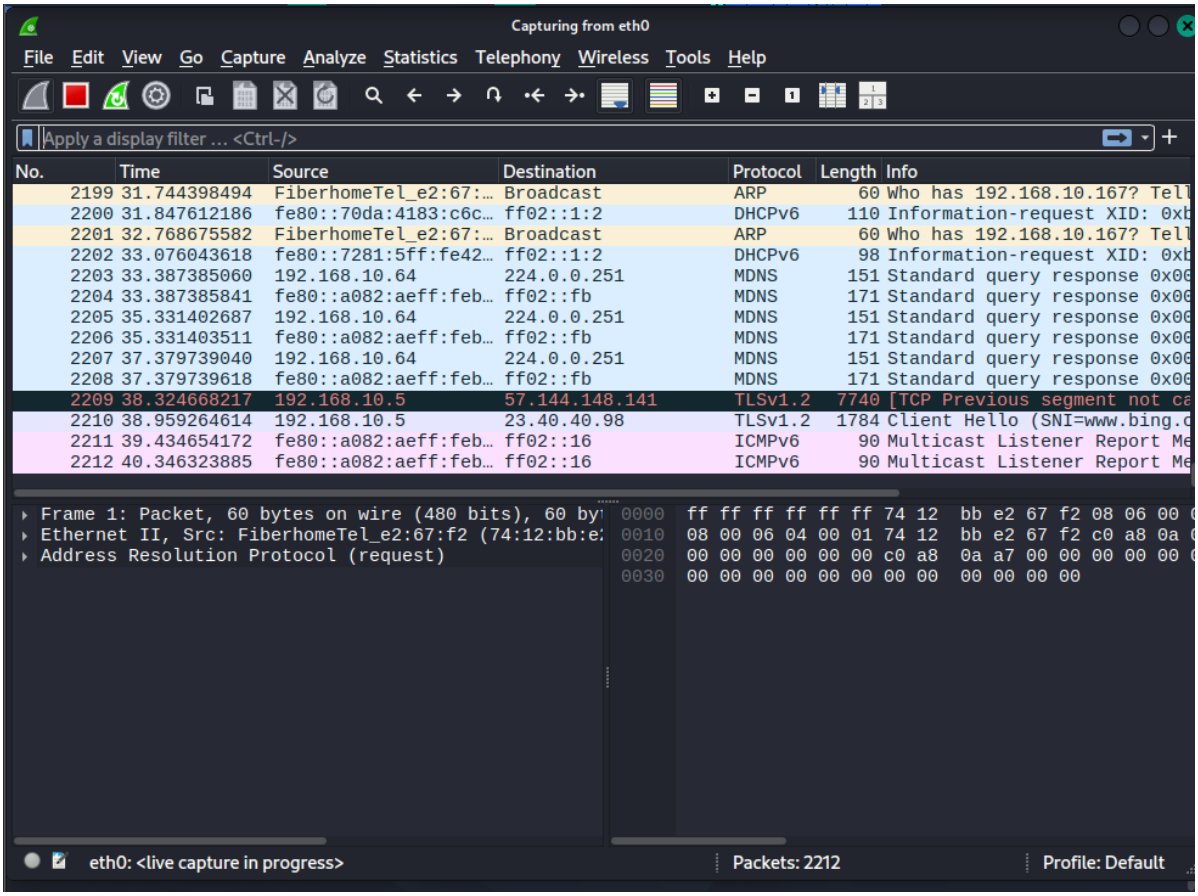


Figure 2 — Step 1 (Wireshark, live capture, no filter): all network traffic, not just the scan.

What this screenshot shows: Wireshark while the Connect capture was still running (status bar: live capture in progress, 2,212 packets at this moment) and before any filter was applied. The list contains general network traffic — ARP requests from the router, DHCPv6, MDNS, and TLS traffic from another machine (192.168.10.5) — mixed together with the scan. This is why the next step matters: without a filter, the scan's packets cannot be counted cleanly.

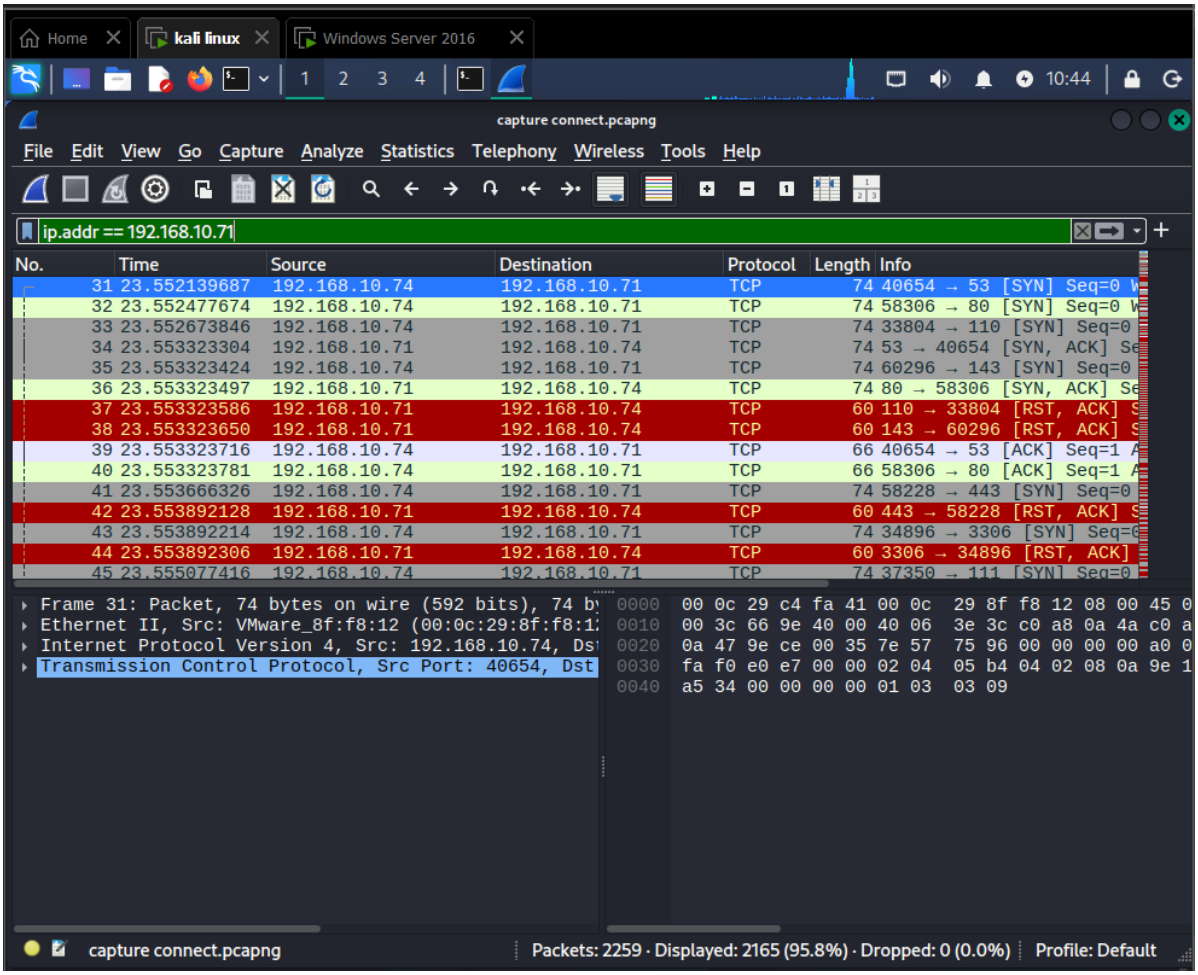


Figure 3 — Step 1 (Wireshark, saved capture_connect.pcapng, filter applied): the Connect scan's packet trail.

What this screenshot shows: The saved Connect capture with the filter ip.addr == 192.168.10.71 applied: 2,259 packets captured in total, 2,165 displayed (95.8%) after filtering. The packets prove how a Connect scan works: for port 80, Kali (192.168.10.74) sends SYN, the target replies SYN/ACK, and Kali completes the handshake with ACK (packets 32, 36 and 40) before the connection is reset. A completed handshake on every port is the loudest possible trail — thousands of packets and a real connection for the target's services to log.

Step 2 — SYN scan (the quiet scan). Capture saved as: capture-syn.pcapng

```
sudo nmap -sS 192.168.10.71 -oN scan-syn.txt
```

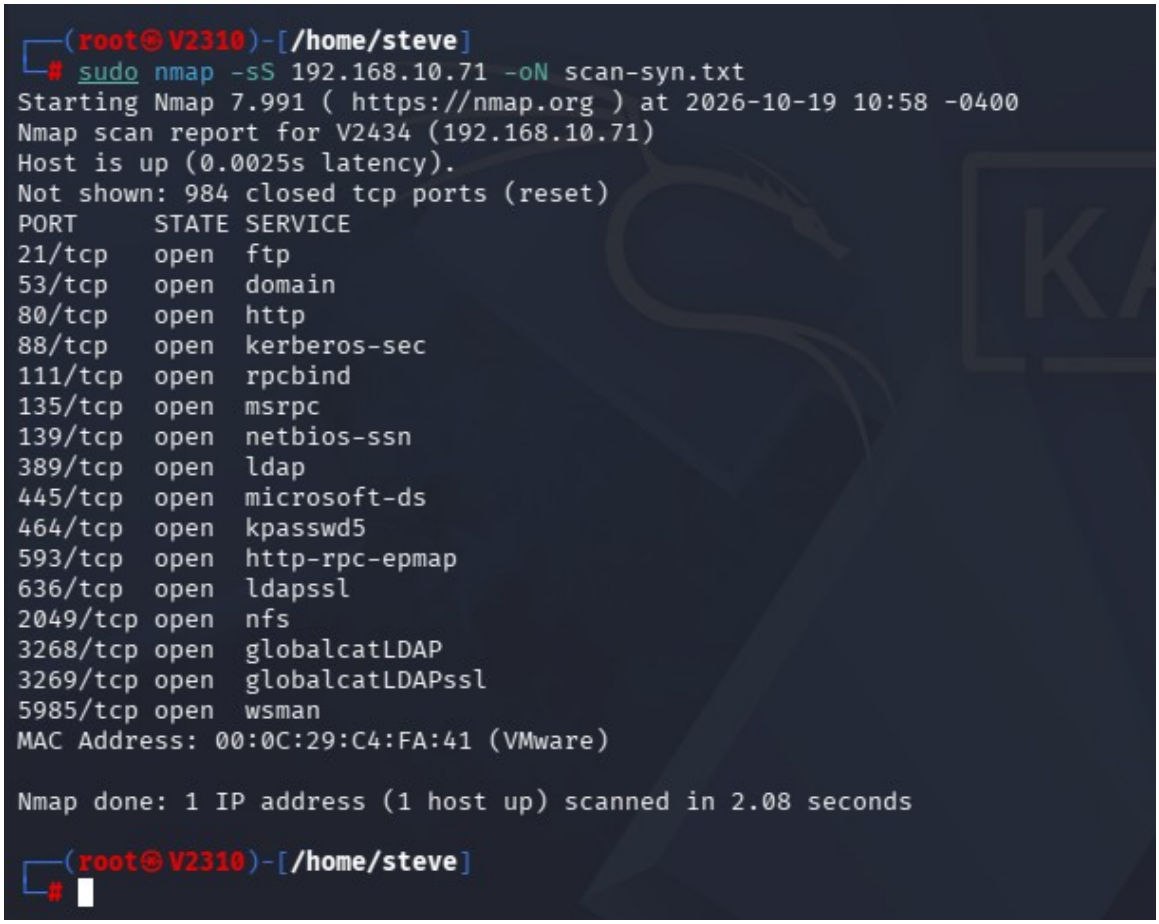


Figure 4 — Step 2 (terminal): the SYN scan result.

What this screenshot shows: The terminal result of the SYN scan. It was run with sudo because a SYN scan builds raw packets, which needs administrator rights. It finished in 2.08 seconds and found the same 16 open TCP ports as the Connect scan. The closed ports are described in the output as closed (reset) — the target answered each closed port with a reset packet, which can be seen happening in the next screenshot.

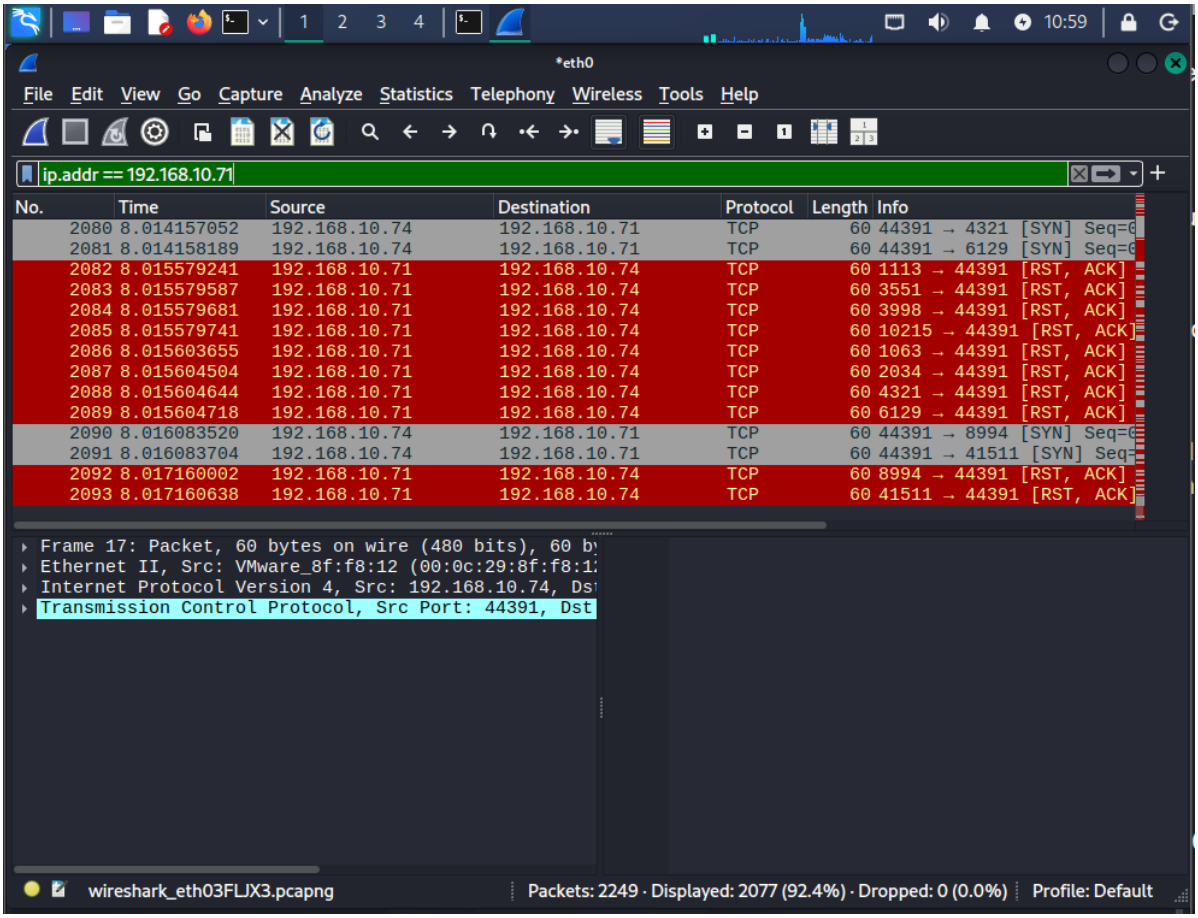


Figure 5 — Step 2 (Wireshark, capture stopped, filter applied): the SYN scan's packet trail.

What this screenshot shows: The stopped SYN capture with the filter applied: 2,249 packets captured in total, 2,077 displayed (92.4%) for the target, and 0 dropped. The visible rows show the SYN pattern: Kali (192.168.10.74) sends [SYN] probes, and the target answers the closed ports immediately with [RST, ACK]. For the 16 open ports the pattern is SYN, SYN/ACK, then a RST from Kali — never the final ACK — so no connection ever completes. The count (2,077) is a little lower than the Connect scan (2,165): the difference is the ACK and completed-connection packets that the SYN scan skips on the open ports.

Step 3 — UDP scan (the slow, silent scan). Capture saved as: capture-udp.pcapng

```
sudo nmap -sU --top-ports 10 192.168.10.71 -oN scan-udp.txt
```

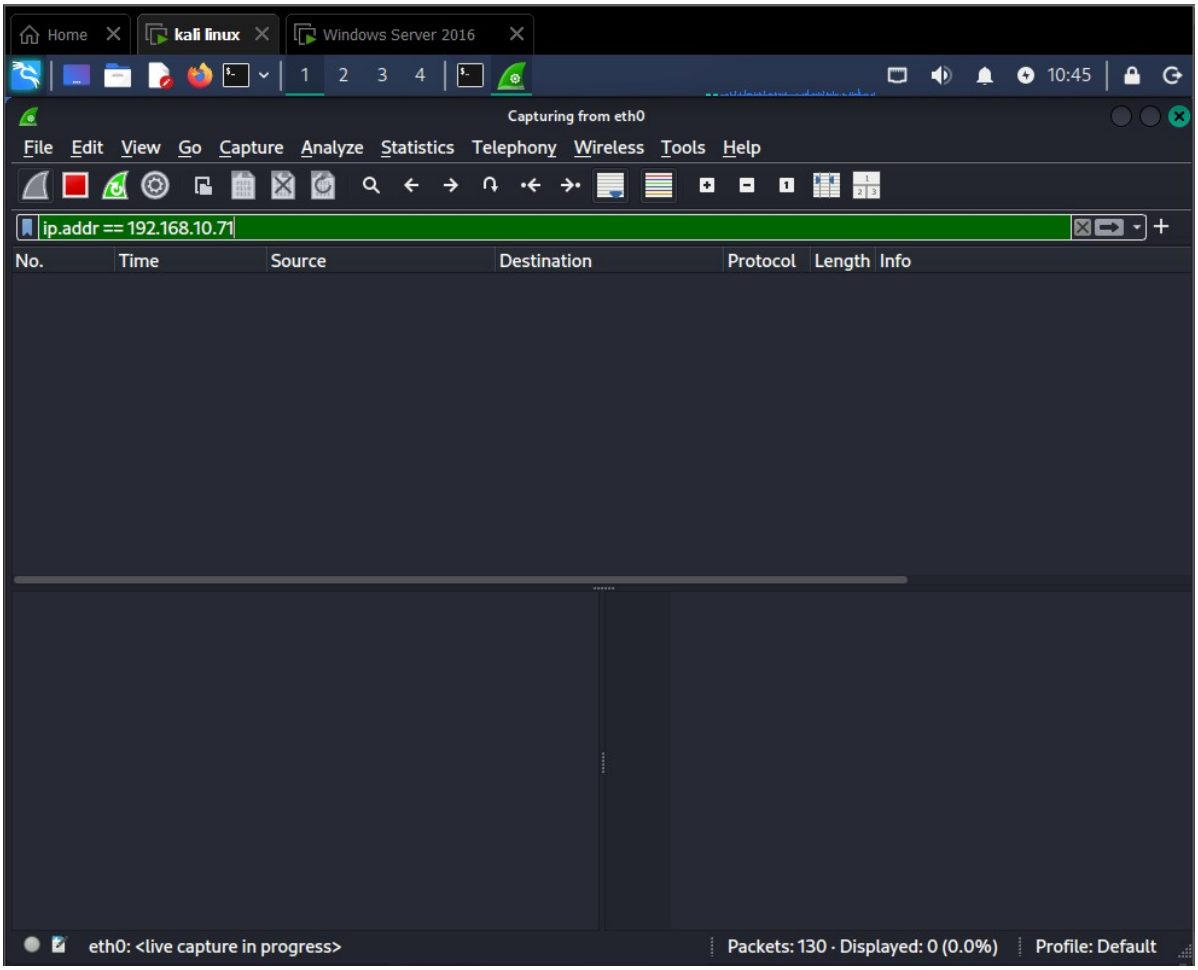


Figure 6 — Step 3 (Wireshark at capture start): filter already set, Displayed 0.

What this screenshot shows: The start of the UDP capture, taken in the correct order: Wireshark is already recording (live capture in progress) and the filter ip.addr == 192.168.10.71 is already typed in, but Displayed shows 0 because the scan has not produced any target traffic yet at this moment. This screenshot proves the method: record first, scan second.

```
┌──(root㉿V2310)-[/home/steve]
└─# sudo nmap -sU --top-ports 10 192.168.10.71 -oN scan-udp.txt

Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-19 10:46 -0400
Nmap scan report for V2434 (192.168.10.71)
Host is up (0.0010s latency).

PORT     STATE         SERVICE
53/udp   open          domain
67/udp   closed        dhcps
123/udp  open          ntp
135/udp  closed        msrpc
137/udp  open          netbios-ns
138/udp  open|filtered netbios-dgm
161/udp  closed        snmp
445/udp  open|filtered microsoft-ds
631/udp  closed        ipp
1434/udp closed        ms-sql-m
MAC Address: 00:0C:29:C4:FA:41 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 5.36 seconds

┌──(root㉿V2310)-[/home/steve]
└─#
```

Figure 7 — Step 3 (terminal): the UDP scan result.

What this screenshot shows: The UDP scan of the 10 most common UDP ports, finished in 5.36 seconds. Three ports answered as open: 53 (domain/DNS), 123 (ntp) and 137 (netbios-ns). Two ports gave no answer at all — 138 (netbios-dgm) and 445 (microsoft-ds) — so Nmap can only report them as open|filtered: with UDP, silence does not prove whether a service is listening or a filter dropped the probe. Five ports (67, 135, 161, 631, 1434) were closed, which the next screenshot shows were answered with ICMP unreachable messages.

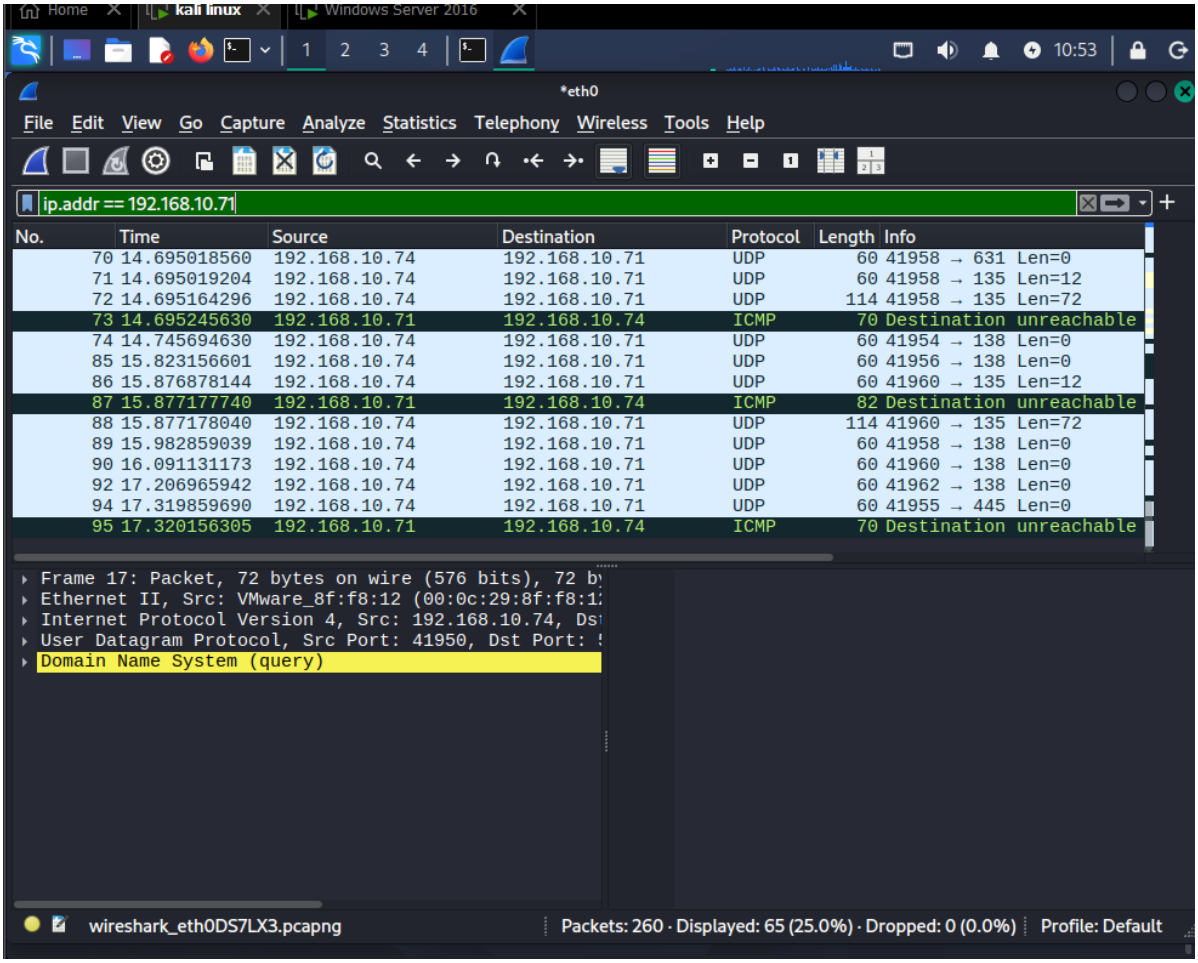


Figure 8 — Step 3 (Wireshark, capture stopped, filter applied): the UDP scan's packet trail.

What this screenshot shows: The stopped UDP capture with the filter applied: 260 packets captured in total, only 65 displayed (25.0%) for the target, and 0 dropped. The trail shows the UDP pattern clearly: Kali sends a UDP probe (for example to ports 631, 135, 138, 445), and every closed port answers with an ICMP Destination unreachable message from the target, while open ports mostly stay silent. 65 packets against the Connect scan's 2,165 is the clearest number in this project: UDP leaves by far the smallest trail.

4. Comparison Table

Packet counts are the Displayed values from Wireshark after applying the filter ip.addr == 192.168.10.71, so all three scans are compared on the same basis.

Scan Type	Command	Packets (Displayed / Total)	Time Taken	Ports Found	Handshake	Noise Level
TCP Connect	nmap -sT	2,165 / 2,259	1.73 s	16 open TCP ports	Full — SYN, SYN/ACK, ACK completed	Loudest
SYN (stealth)	sudo nmap -sS	2,077 / 2,249	2.08 s	16 open TCP ports	Half-open — SYN, SYN/ACK, RST	Quieter
UDP	sudo nmap -sU --top-ports 10	65 / 260	5.36 s	3 open + 2 open|filtered UDP ports (of top 10)	None — UDP has no handshake	Quietest

5. UDP Scan — Port Detail (from Figure 7)

Port	State	Service
53/udp	open	domain
67/udp	closed	dhcps
123/udp	open	ntp
135/udp	closed	msrpc
137/udp	open	netbios-ns
138/udp	open|filtered	netbios-dgm
161/udp	closed	snmp
445/udp	open|filtered	microsoft-ds
631/udp	closed	ipp
1434/udp	closed	ms-sql-m

6. Observations

- The Connect scan produced 2,165 filtered packets — about 33 times more than the UDP scan's 65 — because every one of the 1000 probed ports generates a complete connection attempt or a refusal, and every open port completes a full handshake.

- Evidence of loudness appeared inside the Connect scan's own output: the target reset a connection ('Connection reset by peer'), showing its services were handling — and reacting to — real completed connections. Those connections are what services and firewalls write into their logs.
- The unfiltered capture (Figure 2) shows why the filter step is essential: a raw capture on a shared network also records other machines' traffic (ARP, MDNS, TLS), which would corrupt the packet counts if it were not filtered out.
- The UDP capture shows the two possible UDP answers: a reply from an open service (rare), or an ICMP Destination unreachable from a closed port. No answer at all is ambiguous — Nmap reported those ports as open|filtered (138 and 445).
- The UDP scan was also the slowest of the completed scans (5.36 seconds for only 10 ports, against 1.73 seconds for the Connect scan's 1000 ports), because Nmap must wait for replies or timeouts on every UDP probe.
- The SYN scan's packet count (2,077) is close to the Connect scan's (2,165), and the screenshots explain why: 984 of the 1000 ports are closed, and a closed port costs the same two packets in both scans (SYN, then RST/ACK). The saving appears only on the 16 open ports. So for this target, the SYN scan's real advantage is not the packet count — it is the missing log entries, because no connection ever completed.

7. Analysis — Which Scan Would an IDS Catch First?

An IDS would catch the TCP Connect scan first, and the evidence in this project shows why. It completes the full 3-way handshake on every port, which produced the largest packet trail (2,165 packets) and, more importantly, the largest log footprint: a completed connection is a real event for the target. The web server, SMB service and every other touched service can log it, the firewall can log it, and the target was even seen resetting one of the connections in the scan output. A thousand completed connections in under two seconds is a pattern no monitoring tool can miss.

The UDP scan is the opposite extreme. It left only 65 packets, has no handshake to log, and its open services mostly stayed silent. It is the hardest of the completed scans to detect from logs — but it pays for that quietness: it is slow (waiting for replies that usually never come), and its results are uncertain, as the two open|filtered ports show. Quiet does not mean better; it means quieter and less sure.

The SYN scan sits between the two, and its capture confirms the theory. Its trail (2,077 packets) is slightly smaller than the Connect scan's, and the packets it saves are exactly the telling ones: on open ports there is no final ACK and no completed connection, so the target's applications have nothing to log. An IDS can still detect the pattern — the capture is full of SYNs answered by RSTs — but it has to look at packet patterns rather than simply reading connection logs, which makes the SYN scan meaningfully quieter where it matters most: in the logs.

Conclusion: completed connections are the loudest evidence. Connect (2,165 packets, full handshakes, real log entries) = loudest and caught first. SYN (2,077 packets, half-open, almost no logs) = nearly as many packets, but far quieter in the logs. UDP (65 packets, no handshake) = quietest, but slowest and least reliable.

8. Result

Self-check against the marking criteria:

- Execution (40): all three scans (Connect, SYN and UDP) executed correctly against the lab target, each with its own output file and Wireshark capture.
- Analysis (40): the IDS question is answered with packet-count evidence from all three captures (2,165 vs 2,077 vs 65).
- Captures (20): all three captures are saved and screenshotted at their steps — Connect (2,259 packets), SYN (2,249 packets) and UDP (260 packets).

Deliverables complete: this report, with the comparison table and the one-page analysis, plus the three packet capture files (capture_connect.pcapng, capture-syn.pcapng and capture-udp.pcapng).

— End of Report —